Численные методы решения стационарного уравнения теплопроводности. Отчет

Новикова Вера, 301 группа

Вариант 4.3

Постановка задачи

В данной работе реализовано численное решение краевой задачи для одномерного стационарного уравнения диффузии:

$$\begin{cases} -\frac{d}{dx}\left(p(x)\frac{du}{dx}\right) + q(x)u(x) = f(x), & x \in [0, L],\ L = \pi \\ -p(0)\frac{du}{dx}(0) + \kappa_0 u(0) = g_0, & \kappa_0 = 1, g_0 = 1 \\ u(L) = g_1, & g_1 = 1 \end{cases}$$

$$p(x) = \begin{cases} 1 + \frac{1}{4}\sin(x), & x \in [0; \frac{\pi}{3}), \\ 1 + \frac{1}{3}\sin(x), & x \in \left(\frac{\pi}{3}, \frac{2\pi}{3}\right), \\ 1 + \frac{1}{2}\sin(x), & x \in \left(\frac{2\pi}{3}, \pi\right] \end{cases} \qquad q(x) = \begin{cases} 1, & x \in [0; \frac{\pi}{3}), \\ 2, & x \in \left(\frac{\pi}{3}, \frac{2\pi}{3}\right), \\ 1, & x \in \left(\frac{2\pi}{3}, \pi\right] \end{cases}$$

$$f(x) = 1 - cos(2x)$$

Была построена консервативная схема на неравномерной расчетной сетке и приведен вывод ее погрешности аппроксимации, а также продемонстрирована сходимость метода с помощью метода Рунге.

Вывод порядка погрешности аппроксимации схемы на неравномерной сетке

Решаем уравнение:

$$-\frac{d}{dx}\left(p(x)\frac{du}{dx}\right) + q(x)u(x) = f(x)$$

Перепишем его в виде:

$$-p(x)u''(x) - p'(x)u'(x) + q(x)u(x) - f(x) = 0 \tag{1}$$

Пусть $\{x_i\}$ - узлы сетки. Обозначим

$$x_{i+\frac{1}{2}} = \frac{x_i + x_{i+1}}{2},\ x_{i-\frac{1}{2}} = \frac{x_{i-1} + x_i}{2},\ h_i = x_{i+1} - x_i,\ h_{i-1} = x_i - x_{i-1}$$

Проинтегрируем (1) от $x_{i-1/2}$ до $x_{i+1/2}$:

$$-p(x)\frac{du}{dx}\bigg|_{x_{i-1/2}}^{x_{i+1/2}} + \int_{x_{i-1/2}}^{x_{i+1/2}} \big[q(x)u(x) - f(x)\big]dx = 0$$

Если функция $q(x)$ разрывна, то при интегрировании возникнут проблемы. Для этого приблизим ее некоторым средним значением: $\overline{q_i} = \frac{1}{h}\int_{x_{i-1/2}}^{x_{i+1/2}} q(x)dx$. Тогда $\int qu \approx \overline{q}\int u$. Ищем погрешность аппроксимации решения:

$$-p_{i+1/2}\frac{u_{i+1} - u_i}{h_i} + p_{i-1/2}\frac{u_i - u_{i-1}}{h_{i-1}} + \frac{h_i + h_{i-1}}{2}\big[\overline{q_i}u_i - f_i\big] = 0$$

$$-\frac{2}{h_i + h_{i-1}}\left[p_{i+1/2}\frac{u_{i+1} - u_i}{h_i} - p_{i-1/2}\frac{u_i - u_{i-1}}{h_{i-1}}\right] + q_i u_i - f_i = 0$$

Для этого разложим $u_{i\pm1}$ и $p_{i\pm1/2}$ в ряд Тейлора:

$$u_{i+1} = u_i + h_i\frac{du_i}{dx} + \frac{h_i^2}{2}\frac{d^2u_i}{dx^2} + \frac{h_i^3}{6}\frac{d^3u_i}{dx^3} + \mathrm{O}(h_i^4)$$

$$u_{i-1} = u_i - h_i\frac{du_i}{dx} + \frac{h_i^2}{2}\frac{d^2u_i}{dx^2} - \frac{h_i^3}{6}\frac{d^3u_i}{dx^3} + \mathrm{O}(h_i^4)$$

$$p_{i+1/2} = p_i + \frac{h_i}{2}\frac{dp_i}{dx} + \frac{h_i^2}{8}\frac{d^2p_i}{dx^2} + \frac{h_i^3}{48}\frac{d^3p_i}{dx^3} + \mathrm{O}(h_i^4)$$

$$p_{i-1/2} = p_i - \frac{h_i}{2}\frac{dp_i}{dx} + \frac{h_i^2}{8}\frac{d^2p_i}{dx^2} - \frac{h_i^3}{48}\frac{d^3p_i}{dx^3} + \mathrm{O}(h_i^4)$$

Обозначим $\overline{h_i} = \frac{h_i + h_{i-1}}{2}$, $h = \max\limits_i(h_i)$ Тогда оценим погрешность аппроксимации Ψ как:

$$\Psi = -\frac{1}{\overline{h_i}}\Bigg[\Big(p_i + \frac{h_i}{2}\frac{dp_i}{dx} + \frac{h_i^2}{8}\frac{d^2p_i}{dx^2} + \mathrm{O}(h^3)\Big)\Big(\frac{du_i}{dx} + \frac{h_i}{2}\frac{d^2u_i}{dx^2} + \frac{h_i^2}{6}\frac{d^3u_i}{dx^3} + \mathrm{O}(h^3)\Big) -$$

$$-\Big(p_i - \frac{h_{i-1}}{2}\frac{dp_i}{dx} + \frac{h_{i-1}^2}{8}\frac{d^2p_i}{dx^2} + \mathrm{O}(h^3)\Big)\Big(\frac{du_i}{dx} - \frac{h_{i-1}}{2}\frac{d^2u_i}{dx^2} + \frac{h_{i-1}^2}{6}\frac{d^3u_i}{dx^3} + \mathrm{O}(h^3)\Big)\Bigg] + q_iu_i - f_i =$$

$$= -\frac{1}{\overline{h_i}}\Bigg[p_i\Big(\overline{h_i}\frac{d^2u_i}{dx^2} + \frac{h_i^2 - h_{i-1}^2}{6}\frac{d^3u_i}{dx^3} + \mathrm{O}(h^3)\Big) + \frac{dp_i}{dx}\Big(\overline{h_i}\frac{du_i}{dx} + \frac{h_i^2 - h_{i-1}^2}{4}\frac{d^2u_i}{dx^2} + \mathrm{O}(h^3)\Big) +$$

$$+\frac{d^2p_i}{dx^2}\Big(\frac{h_i^2 - h_{i-1}^2}{8}\frac{du_i}{dx} + \mathrm{O}(h^3)\Big)\Bigg] + q_iu_i - f_i = -p_i\frac{d^2u_i}{dx^2} - p_i\frac{h_i - h_{i-1}}{3}\frac{d^3u_i}{dx^3} -$$

$$-\frac{dp_i}{dx}\frac{du_i}{dx} - \frac{dp_i}{dx}\frac{h_i - h_{i-1}}{2}\frac{d^2u_i}{dx^2} - \frac{d^2p_i}{dx^2}\frac{h_i - h_{i-1}}{4}\frac{du_i}{dx} + q_iu_i - f_i + \mathrm{O}(h^2) =$$

$$= \{(1)\} = (h_i - h_{i-1})(-\frac{p_i}{3}\frac{d^3u_i}{dx^3} - \frac{1}{2}\frac{dp_i}{dx}\frac{d^2u_i}{dx^2} - \frac{1}{4}\frac{dp_i}{dx}\frac{du_i}{dx}) + \mathrm{O}(h^2) = C(h_i - h_{i-1}) + \mathrm{O}(h^2)$$

Если $h_i - h_{i-1} = \mathrm{O}(h^2)$, то общая погрешность аппроксимации порядка 2.
Ищем погрешность аппроксимации граничного условия:

$$-p(0)\frac{du}{dx}(0) + \kappa_0 u(0) = g_0$$

Проинтегрируем (1) от x_0 до $x_{1/2}$:

$$-p(x)\frac{du}{dx}\bigg|_{x_0}^{x_{1/2}} + \int_{x_0}^{x_{1/2}} [q(x)u(x) - f(x)]dx$$

С учетом:

$$\int_0^{x_{1/2}} [q(x)u(x) - f(x)]\, dx \approx (q(0)u(0) - f(0))\frac{h_1}{2}$$

Получаем:

$$-p_{1/2}u'_{1/2} + \frac{h_1}{2}\big(q(0)u(0) - f(0)\big) = -p(0)u'(0) = g_0 - \kappa_0 u(0)$$

Разложим по Тейлору:

$$-\big(p(0) + \frac{h_1}{2}p'(0) + \mathrm{O}(h^2)\big)\big(u'(0) + \frac{h_1}{2}u''(0) + \mathrm{O}(h^2)\big) + \frac{h_1}{2}\big(q(0)u(0) - f(0)\big) = g_0^h - \kappa_0 u(0)$$

$$-p(0)u'(0) - \frac{h_1}{2}p'(0)u'(0) - \frac{h_1}{2}p(0)u''(0) + \frac{h_1}{2}(q(0)u(0) - f(0)) + \kappa_0 u(0) + O(h^2) = g_0^h$$

Погрешность

$$\Psi = g_0 - g_0^h = g_0 + p(0)u'(0) + \frac{h_1}{2}p'(0)u'(0) + \frac{h_1}{2}p(0)u''(0) - \frac{h_1}{2}\big(q(0)u(0) - f(0)\big) - \kappa_0 u(0) + O(h^2)$$

В силу граничных условий и равенства уравнения погрешность $\Psi = O(h^2)$ (остальные слагаемые сокращаются).
Итак, для того, чтобы порядок аппроксимации схемы был равен 2, необходимо выполнение условия $h_i - h_{i-1} = O(h^2)$, где $h = \max\limits_i(h_i)$.

Построение неравномерной сетки

Обозначим $\phi(t) : [0, 1] \to [\alpha, \beta]$ - вспомогательная функция, по которой будем выбирать узлы сетки. На ϕ наложим следующие ограничения:

1. $\phi(t) \in C^2[0, 1]$
2. $\phi'(t) > 0$

Покажем, что при $\phi(t) = \alpha + (\beta - \alpha)\frac{e^{ct}-1}{e^c-1}$ построенная неравномерная сетка $\{x_i\} = \{\phi(t_i)\}, t_i = i\theta, \theta = \frac{1}{n}, i = \overline{1,n}$ удовлетворяет условию для 2 порядка аппроксимации схемы.

$$\phi(t) \in C^2[0,1], \quad \phi'(t) = (\beta - \alpha)\frac{ce^{ct}}{e^c - 1} > 0$$

$$h_{i+1} = x_{i+1} - x_i = \phi(t_{i+1}) - \phi(t_i) = (\beta - \alpha)\frac{e^{c(i+1)\theta} - e^{ci\theta}}{e^c - 1} = (\beta - \alpha)e^{ci\theta}\frac{e^{c\theta} - 1}{e^c - 1}$$

$$h_i = x_i - x_{i-1} = \phi(t_i) - \phi(t_{i-1}) = (\beta - \alpha)\frac{e^{ci\theta} - e^{c(i-1)\theta}}{e^c - 1} = (\beta - \alpha)e^{ci\theta}\frac{1 - e^{-c\theta}}{e^c - 1}$$

$$\frac{h_{i+1} - h_i}{h_i^2} = \frac{\beta - \alpha}{e^c - 1}e^{ci\theta}(e^{c\theta} - 2 + e^{-c\theta})\left(\frac{e^c - 1}{(\beta - \alpha)e^{ci\theta}(1 - e^{-c\theta})}\right)^2 = \frac{e^{-c\theta}(e^{c\theta} - 1)^2(e^c - 1)}{(\beta - \alpha)e^{ci\theta}(1 - e^{-c\theta})^2}$$

$$\frac{h_{i+1} - h_i}{h_i^2} \to const, \ \theta \to 0$$

Итак, для построения сетки будем использовать вспомогательную функцию (положим $\alpha = 0, \beta = \pi, c = 1$):

$$\phi(t) = \pi\frac{e^t - 1}{e - 1}$$

Решение СЛАУ

В итоге получили следующую схему:

$$\begin{cases} u_{i-1}\left(-\frac{p_{i-1/2}}{h_i}\right) + u_i\left(\frac{p_{i+1/2}}{h_{i+1}} + \frac{p_{i-1/2}}{h_i} + q_i\overline{h_i}\right) + u_{i+1}\left(-\frac{p_{i-1/2}}{h_i}\right) = f_i\overline{h_i} \quad i = \overline{1,n} \\ u_0\left(\frac{p_{1/2}}{h_1} + q_0\frac{h_1}{2} + 1\right) + u_1\left(-\frac{p_{1/2}}{h_1}\right) = f_0\frac{h_1}{2} + 1 \\ u_n = 1 \end{cases}$$

Она задает СЛАУ с трехдиагональной матрицей, поэтому имеет смысл решать ее методом прогонки. Если система уравнений имеет вид:

$$\begin{cases} b_1x_1 + c_1x_2 = d_1 \\ a_ix_{i-1} + b_ix_i + c_ix_{i+1} = d_i, \quad i = 2, \dots, n-1 \\ a_nx_{n-1} + b_nx_n = d_n \end{cases} \tag{2}$$

То прямой ход (вычисление прогоночных коэффициентов α_i и β_i) выглядит следующим образом:

1. Для первой строки ($i = 1$):

$$y_1 = b_1, \quad \alpha_1 = \frac{-c_1}{y_1}, \quad \beta_1 = \frac{d_1}{y_1} \tag{3}$$

2. Для строк $i = 2, \dots, n-1$:

$$y_i = b_i + a_i\alpha_{i-1}, \quad \alpha_i = \frac{-c_i}{y_i}, \quad \beta_i = \frac{d_i - a_i\beta_{i-1}}{y_i} \tag{4}$$

3. Для последней строки ($i = n$):

$$y_n = b_n + a_n\alpha_{n-1}, \quad \beta_n = \frac{d_n - a_n\beta_{n-1}}{y_n} \tag{5}$$

А обратный ход:

1. Последняя компонента решения:

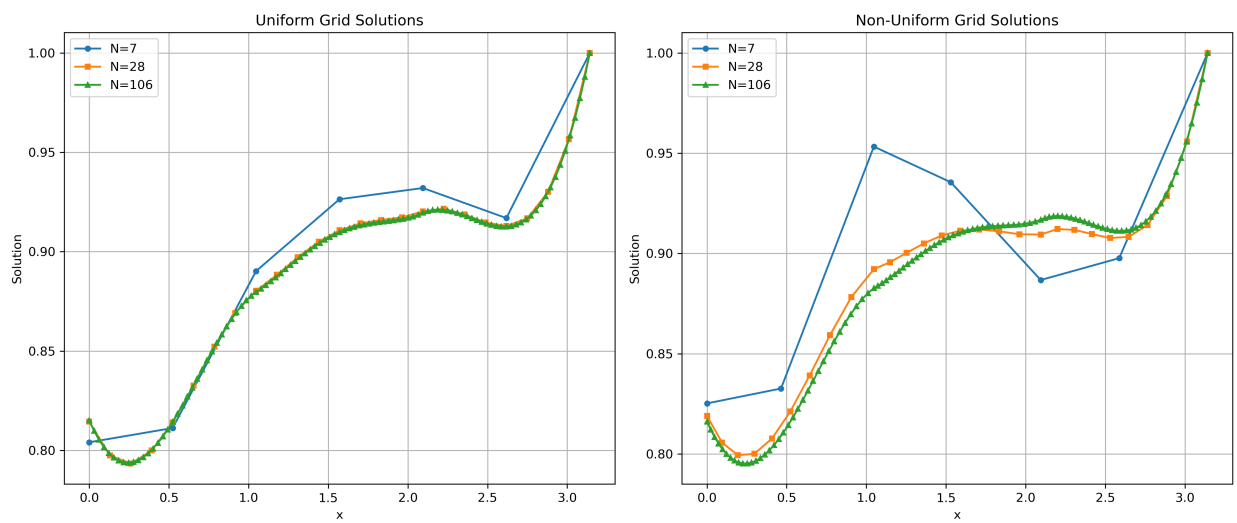
$$x_n = \beta_n \tag{6}$$

2. Остальные компоненты (для $i = n-1, \ldots, 1$):

$$x_i = \alpha_i x_{i+1} + \beta_i \tag{7}$$

Будем строить сетку так, чтобы точки разрыва $\frac{\pi}{3}, \frac{2\pi}{3}$ попадали в узлы.

Результаты



Как можно видеть, решение сходится. Это подтверждается численно. Для демонстрации сходимости метода рассмотрим норму:

$$||u_n - u_{2n}|| = \max_n |u_n - u_{2n}|$$

Uniform grid

Nodes N	Nodes 2N-1	Max Error
7	13	1.259141e-02
43	85	2.461324e-04
79	157	7.137685e-05
115	229	3.340919e-05
151	301	1.929690e-05
187	373	1.254993e-05
223	445	8.809465e-06
259	517	6.522631e-06
295	589	5.022916e-06
331	661	3.986862e-06
367	733	3.241052e-06
403	805	2.686610e-06
439	877	2.263093e-06
475	949	1.932408e-06
511	1021	1.669214e-06
547	1093	1.456367e-06
583	1165	1.281764e-06
619	1237	1.136774e-06
655	1309	1.015076e-06
691	1381	9.119252e-07
727	1453	8.237194e-07
763	1525	7.477319e-07
799	1597	6.817837e-07
835	1669	6.241736e-07
871	1741	5.736168e-07
907	1813	5.289333e-07
943	1885	4.892546e-07
979	1957	4.539302e-07

Рис. 1: На равномерной сетке

Non-uniform grid

Nodes N	Nodes 2N-1	Max Error
7	13	4.236190e-02
43	85	3.992490e-03
79	157	2.060058e-03
115	229	1.386835e-03
151	301	1.045024e-03
187	373	8.383258e-04
223	445	6.998691e-04
259	517	6.006544e-04
295	589	5.260720e-04
331	661	4.679628e-04
367	733	4.214125e-04
403	805	3.832845e-04
439	877	3.514829e-04
475	949	3.245538e-04
511	1021	3.014572e-04
547	1093	2.814294e-04
583	1165	2.638967e-04
619	1237	2.484204e-04
655	1309	2.346587e-04
691	1381	2.223417e-04
727	1453	2.112531e-04
763	1525	2.012179e-04
799	1597	1.920929e-04
835	1669	1.837596e-04
871	1741	1.761193e-04
907	1813	1.690889e-04
943	1885	1.625982e-04
979	1957	1.565874e-04

Рис. 2: На неравномерной сетке

Видим, что с увеличением числа узлов (уменьшением шага сетки) ошибка становится меньше.
Для демонстрации сходимости схемы воспользуемся методом Рунге:

$$\log_2\left(\frac{\|u_n - u_{2n}\|}{\|u_{2n} - u_{4n}\|}\right) = \log_2\left(\frac{C(h^p - (\frac{h}{2})^p) + O(h^2)}{C((\frac{h}{2})^p - (\frac{h}{4})^p) + O(h^2)}\right) = p$$

```
Uniform grid
   Nodes N        Convergence order
-----------------------------------
         7                 2.049409
        13                 2.012485
        19                 2.004464
        25                 1.999837
        31                 1.998182
        37                 1.998644
        43                 1.999875
        49                 2.000572
        55                 2.000620
        61                 2.000502
        67                 2.000414
        73                 2.000348
        79                 2.000297
        85                 2.000226
        91                 2.000007
        97                 1.999838
Non-uniform grid
   Nodes N        Convergence order
-----------------------------------
         7                 1.943236
        19                 2.009765
        28                 1.855134
        32                 1.890613
        41                 1.807287
```

Получился $p = 2$ порядок сходимости на равномерной сетке, на неравномерной в целом тоже, но из-за неравномерности сетки невозможно точно определить порядок аппроксимации.

Код программы

```cpp
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <fstream>
#include <iomanip>
using namespace std;

void write_to_file(const string& filename, const vector<double>& x,
    const vector<double>& y) {
```

```
    ofstream outfile(filename);
    if (!outfile.is_open()) {
        cerr << "Error␣opening␣file:␣" << filename << endl;
        return;
    }
    outfile << "x,y\n";
    for (size_t i = 0; i < x.size(); ++i) {
        outfile << x[i] << "," << y[i] << "\n";
    }
    outfile.close();
}

class HeatEquationSolver {
private:
    const double L = M_PI;
    const double kappa0 = 1.0;
    const double g0 = 1.0;
    const double g1 = 1.0;
    int n;
    std::vector<double> x;
    std::vector<double> h;
    bool non_uniform_grid = false;

    double phi(double t) const {
        return L * (exp(t) - 1.0) / (exp(1.0) - 1.0);
    }

    double p(double x) const {
        if (x == M_PI/3.0) return 1.0 + sin(x)/8.0 + sin(x)/6.0;
        else if (x == 2.0*M_PI/3.0) return 1.0 + sin(x)/4.0 + sin(x)
            /6.0;
        if (x < M_PI/3.0) return 1.0 + 0.25*sin(x);
        else if (x < 2.0*M_PI/3.0) return 1.0 + sin(x)/3.0;
        else return 1.0 + 0.5*sin(x);
    }

    double q(double x) const {
        if (x == M_PI/3.0 || x == 2.0*M_PI/3.0) return 1.5;
        else if (x < M_PI/3.0) return 1.0;
        else if (x < 2.0*M_PI/3.0) return 2.0;
        else return 1.0;
    }

    double f(double x) const {
        return 1.0 - cos(2.0*x);
    }

```

```
public:
    HeatEquationSolver(int num_points, bool _non_uniform_grid = false) : n(num_points) {
        non_uniform_grid = _non_uniform_grid;
        x.resize(n);
        h.resize(n);
        if (non_uniform_grid == false) {
            int block = (n + 2) / 3;
            double step = M_PI / 3 / (block - 1);
            for (int i = 0; i < block; ++i) {
                x[i] = step * i;
            }
            for (int i = 1; i < block; ++i) {
                x[block - 1 + i] = M_PI/3 + step * i;
            }
            for (int i = 1; i < block; ++i) {
                x[2 * block - 2 + i] = 2 * M_PI/3 + step * i;
            }
        }
        else {
            const double t_pi3 = log(1.0 + (M_E - 1.0) / 3.0);
            const double t_2pi3 = log(1.0 + 2.0 * (M_E - 1.0) / 3.0);
            if (n < 7) {
                cout << "Invalid amount of grid points" << endl;
                exit(1);
            } else {
                int points_between = n / 3;
                int counter = 0;
                for (int i = 0; i < points_between; ++i) {
                    double t = t_pi3 * static_cast<double>(i) / points_between;
                    x[counter] = phi(t);
                    counter++;
                }
                for (int i = 0; i < points_between; ++i) {
                    double t = t_pi3 + (t_2pi3 - t_pi3) * static_cast<double>(i) / points_between;
                    x[counter] = phi(t);
                    counter++;
                }
                for (int i = 0; i < points_between; ++i) {
                    double t = t_2pi3 + (1.0 - t_2pi3) * static_cast<double>(i) / points_between;
                    x[counter] = phi(t);
                    counter++;
                }
                x.back() = M_PI;
```

```
            }
        }
        h[0] = 0;
        for (int i = 1; i < n; ++i) {
            h[i] = x[i] - x[i-1];
        }
    }

    std::vector<double> solve() {
        std::vector<std::vector<double>> A(n, std::vector<double>(n, 0.0));
        std::vector<double> b(n, 0.0);
        for (int i = 1; i < n-1; ++i) {
            double p_im12 = p((x[i-1] + x[i]) / 2.0);
            double p_ip12 = p((x[i] + x[i+1]) / 2.0);
            double q_i = q(x[i]);
            double h_i = h[i];
            double h_ip1 = h[i+1];
            double h_avg = (h_i + h_ip1) / 2.0;

            A[i][i-1] = -p_im12 / h_i;
            A[i][i] = p_ip12 / h_ip1 + p_im12 / h_i + q_i * h_avg;
            A[i][i+1] = -p_ip12 / h_ip1;
            b[i] = f(x[i]) * h_avg;
        }

        double p_12 = p((x[0] + x[1]) / 2.0);
        double q_0 = q(x[0]);
        A[0][0] = p_12 / h[1] + q_0 * h[1] / 2.0 + kappa0;
        A[0][1] = -p_12 / h[1];
        b[0] = f(x[0]) * h[1] / 2.0 + g0;

        A[n-1][n-1] = 1.0;
        b[n-1] = g1;

        return thomas_algorithm(A, b);
    }
    const std::vector<double>& get_grid() const { return x; }

private:
    std::vector<double> thomas_algorithm(const std::vector<std::vector<double>>& A,
                                         const std::vector<double>& b) {
        int n = A.size();
        std::vector<double> alpha(n, 0.0);
        std::vector<double> beta(n, 0.0);
```

```
        std::vector<double> x(n, 0.0);
        double y = A[0][0];

        alpha[0] = -A[0][1] / y;
        beta[0] = b[0] / y;
        for (int i = 1; i < n - 1; i++) {
            y = A[i][i] + A[i][i - 1] * alpha[i - 1];
            alpha[i] = -A[i][i + 1] / y;
            beta[i] = (b[i] - A[i][i - 1] * beta[i - 1]) / y;
        }
        x[n - 1] = (b[n - 1] - A[n - 1][n - 2] * beta[n - 2]) / (A[n
           - 1][n - 1] + A[n - 1][n - 2] * alpha[n - 2]);
        for (int i = n - 2; i >= 0; i--) {
            x[i] = alpha[i] * x[i + 1] + beta[i];
        }

        return x;
    }
};

int main()
{
    // 7, 28, 106
    HeatEquationSolver solver7(7);
    HeatEquationSolver solver28(28);
    HeatEquationSolver solver106(106);

    vector<double> res7 = solver7.solve();
    vector<double> res28 = solver28.solve();
    vector<double> res106 = solver106.solve();

    vector<double> grid7 = solver7.get_grid();
    vector<double> grid28 = solver28.get_grid();
    vector<double> grid106 = solver106.get_grid();

    HeatEquationSolver solver7NU(7, true);
    HeatEquationSolver solver28NU(28, true);
    HeatEquationSolver solver106NU(106, true);

    vector<double> res7NU = solver7NU.solve();
    vector<double> res28NU = solver28NU.solve();
    vector<double> res106NU = solver106NU.solve();

    vector<double> grid7NU = solver7NU.get_grid();
    vector<double> grid28NU = solver28NU.get_grid();
    vector<double> grid106NU = solver106NU.get_grid();

```

```
    write_to_file("uniform_grid7.csv", grid7, res7);
    write_to_file("uniform_grid28.csv", grid28, res28);
    write_to_file("uniform_grid106.csv", grid106, res106);

    write_to_file("nonuniform_grid7.csv", grid7NU, res7NU);
    write_to_file("nonuniform_grid28.csv", grid28NU, res28NU);
    write_to_file("nonuniform_grid106.csv", grid106NU, res106NU);

    cout << "Uniform grid" << endl;
    cout << setw(10) << "Nodes N"
         << setw(15) << "Nodes 2N-1"
         << setw(20) << "Max Error"
         << endl;
    cout << string(45, '-') << endl;

    cout << scientific << setprecision(6);

    for (int i = 7; i < 1000; i += 36) {
        int N = i;
        HeatEquationSolver solver1(N);
        HeatEquationSolver solver2(2*N - 1);

        vector<double> a = solver1.solve();
        vector<double> b = solver2.solve();
        vector<double> x1 = solver1.get_grid();
        vector<double> x2 = solver2.get_grid();

        double max1 = 0;
        for (int i = 0, j = 0; i < N && j < 2*N-1; ++i, j += 2) {
            if (fabs(x1[i] - x2[j]) < 1e-10) {
                double diff = fabs(a[i] - b[j]);
                if (diff > max1) max1 = diff;
            }
        }
        cout << setw(10) << N
             << setw(15) << 2*N-1
             << setw(20) << max1
             << endl;
    }

    cout << "Non-uniform grid" << endl;
    cout << setw(10) << "Nodes N"
         << setw(15) << "Nodes 2N-1"
         << setw(20) << "Max Error"
         << endl;
    cout << string(45, '-') << endl;

```

```
    cout << scientific << setprecision(6);

    for (int i = 7; i < 1000; i += 36) {
        int N = i;
        HeatEquationSolver solver1(N, true);
        HeatEquationSolver solver2(2*N - 1, true);

        vector<double> a = solver1.solve();
        vector<double> b = solver2.solve();
        vector<double> x1 = solver1.get_grid();
        vector<double> x2 = solver2.get_grid();

        double max1 = 0;
        for (int i = 0, j = 0; i < N && j < 2*N-1; ++i, j += 2) {
            if (fabs(x1[i] - x2[j]) < 1e-10) {
                double diff = fabs(a[i] - b[j]);
                if (diff > max1) max1 = diff;
            }
        }
        cout << setw(10) << N
             << setw(15) << 2*N-1
             << setw(20) << max1
             << endl;
    }
    cout << "Uniform grid" << endl;
    cout << setw(10) << "Nodes N"
         << setw(25) << "Convergence order"
         << endl;
    cout << string(35, '-') << endl;

    cout << fixed << setprecision(6);

    for (int N = 7; N < 100; N += 6) {
        HeatEquationSolver solver1(N);
        HeatEquationSolver solver2(2*N-1);
        HeatEquationSolver solver3(4*N-3);

        vector<double> a = solver1.solve();
        vector<double> b = solver2.solve();
        vector<double> c = solver3.solve();

        vector<double> x1 = solver1.get_grid();
        vector<double> x2 = solver2.get_grid();
        vector<double> x3 = solver3.get_grid();

        double max1 = 0;
        double max2 = 0;
```

```
        for (int i = 0, j = 0; i < N && j < 2*N-1; ++i, j += 2) {
            if (fabs(x1[i] - x2[j]) < 1e-10) {
                double diff = fabs(a[i] - b[j]);
                if (diff > max1) max1 = diff;
            }
        }
        for (int i = 0, j = 0; i < 2*N-1 && j < 4*N-3; i += 1, j +=
           2) {
            if (fabs(x2[i] - x3[j]) < 1e-10) {
                double diff = fabs(b[i] - c[j]);
                if (diff > max2) max2 = diff;
            }
        }

        double convergence_order = (max1 < 1e-12 || max2 < 1e-12) ? 0
            : log2(max1/max2);

        cout << setw(10) << N
             << setw(25) << convergence_order
             << endl;
    }

    cout << "Non-uniform grid" << endl;
    cout << setw(10) << "Nodes N"
         << setw(25) << "Convergence order"
         << endl;
    cout << string(35, '-') << endl;

    cout << fixed << setprecision(6);

    vector<int> test_nodes = {7, 19, 28, 32, 41};

    for (int N : test_nodes) {
        HeatEquationSolver solver1(N, true);
        HeatEquationSolver solver2(2*N-1, true);
        HeatEquationSolver solver3(4*N-3, true);

        vector<double> a = solver1.solve();
        vector<double> b = solver2.solve();
        vector<double> c = solver3.solve();

        vector<double> x1 = solver1.get_grid();
        vector<double> x2 = solver2.get_grid();
        vector<double> x3 = solver3.get_grid();

        double max1 = 0;
        double max2 = 0;
```

```
        for (int i = 0, j = 0; i < N && j < 2*N-1; ++i, j += 2) {
            if (fabs(x1[i] - x2[j]) < 1e-10) {
                double diff = fabs(a[i] - b[j]);
                if (diff > max1) max1 = diff;
            }
        }
        for (int i = 0, j = 0; i < 2*N-1 && j < 4*N-3; i += 1, j +=
           2) {
            if (fabs(x2[i] - x3[j]) < 1e-10) {
                double diff = fabs(b[i] - c[j]);
                if (diff > max2) max2 = diff;
            }
        }

        double convergence_order = (max1 < 1e-12 || max2 < 1e-12) ? 0
            : log2(max1/max2);

        cout << setw(10) << N
             << setw(25) << convergence_order
             << endl;
    }

    return 0;
}
```